



AccumulatR: Structured Evidence-Accumulation Models for Choice and Response-Time Data in R

Niek Stevenson

Independent researcher

Abstract

Choice and response-time data are often modeled with evidence-accumulation models in which latent finishing times determine both the selected response and its latency. Many applications require structured variants of this idea: responses may depend on pooled channels, ordered stages, inhibitory processes, latent components, shared absence events, or logical combinations of several latent completions. **AccumulatR** is an R/C++ package for specifying, simulating, and evaluating likelihoods for structured evidence-accumulation models. Its central object is a race architecture rather than a single named response-time distribution or estimation routine. Users declare accumulators, pools, outcome rules, components, triggers, and mixture settings in R; the finalized model object is then used for simulation, data preparation, response-probability calculation, and likelihood evaluation. Repeated numerical work is delegated to compiled code via **Rcpp**. This paper describes the software design, the mathematical representation used by the likelihood evaluator, and reproducible examples covering basic races, pooled outcomes, guarded rules, mixtures, and ranked responses.

Keywords: evidence accumulation, response times, race models, choice models, R.

1. Introduction

Psychologists study decision-making using choice outcomes and the time at which that choice was made. A model for such data must therefore explain two things at once: which response occurred and when it occurred. Evidence-accumulation models (EAMs) do this by introducing latent processes that evolve over time until a finishing condition is met. The observed response is a label attached to that finishing event, and the response time is its latency.

For many data sets, a standard readout is sufficient: one process reaches one of two boundaries, or the first accumulator to finish determines the response. Other experiments make the readout part of the hypothesis. A response may be produced by the first member of a pool,

by the completion of several processes, by a target process that must finish before an inhibiting process, by a later stage that can start only after an earlier one, or by a latent processing component selected on that trial. These are substantive claims about cognitive architecture, but they are also statistical claims. Changing the readout changes the event whose probability is evaluated and therefore changes the likelihood of the observed choice and response time.

The current software ecosystem is organized mainly around named distributions and fitting workflows. Packages such as **RWiener**, **fddm**, **ddModel**, and **ream** provide distributional calculations for DDM or two-threshold models (Wabersich and Vandekerckhove 2014; Foster, Singmann, and Zeileis 2024; Lin 2025a; Hartmann and Murrow 2025). Packages such as **rtdists**, **lbaModel**, **CircularDDM**, **dynConfR**, **eam**, **ssms**, and **SequentialSamplingModels.jl** provide density, simulation, likelihood, or prediction functions for accumulator, race, continuous-response, confidence, or broader sequential-sampling model families (Singmann, Brown, Gretton, Heathcote, Voss, Voss, and Terry 2026; Lin 2025b; Lin, Heathcote, and Kvam 2018; Hellmann and Rausch 2025; Zhu and Yang 2026; Fengler, Govindarajan, Chen, and Frank 2021; Fernandez, Makowski, and Fisher 2024). Estimation systems such as **fast-dm**, **PyDDM**, **PyBEAM**, **dRiftDM**, **DMCfun**, **diffIRT**, **RegDDM**, **DMC/ggdmc**, **EMC2**, **pmwg**, **brms**, **bmm**, **HDDM**, and **HSSM** provide optimization, Bayesian, hierarchical, regression, simulator-based, or amortized-likelihood workflows (Voss and Voss 2007; Shinn, Lam, and Murray 2020; Murrow and Holmes 2024; Koob, Richter, and Janczyk 2026; Mackenzie and Dudschig 2024; Molenaar, Tuerlinckx, and van der Maas 2015; Jin 2025; Heathcote, Lin, Reynolds, Strickland, Gretton, and Matzke 2019; Lin and Heathcote 2025; Stevenson, Donzallaz, Innes, Forstmann, Matzke, and Heathcote 2024; Cooper, Innes, Kuhne, Cavallaro, Gunawan, Hawkins, Brown, and Stevenson 2024; Bürkner 2017; Popov, Frischkorn, Li, and Buerkner 2026; Wiecki, Sofer, and Frank 2013; Fengler, Xu, Bera, Omar, and Frank 2026). The resulting ecosystem is valuable, but its reusable objects are usually distributions, simulators, fitted models, or sampling workflows. The readout architecture is typically fixed by the named model family. When that architecture is the object to be varied, the user often has to force the hypothesis into an existing readout or write a custom simulator and likelihood.

This distinction has a long history in reaction-time theory. Serial-parallel systems, critical-path models, and queueing-network accounts all treat process architecture as part of the model rather than as verbal interpretation after fitting (Schweickert 1978; Townsend and Wenger 2004; Liu 2007). Race models are a useful way to bring that idea into evidence accumulation because they represent a trial by several latent finishing times (Heathcote and Matzke 2022). The usual winner-takes-all race is then only the simplest order constraint. Pooled, staged, inhibited, delayed, mixed, or ranked responses can be represented by different constraints on the same kind of latent objects.

AccumulatR addresses this gap by making the race architecture a reusable software object. A model specification contains the latent accumulators and the rules that map their finishing times to observed responses: pools, logical outcome rules, triggers, components, mixtures, onsets, and ranked readouts. The same finalized object is used for simulation, response-probability calculation, data preparation, and likelihood evaluation. Internally, the evaluator lowers outcome rules to order constraints over latent finishing times and reuses compiled model state across parameter evaluations. The package therefore provides a model-evaluation layer for structured evidence-accumulation models; it does not try to replace optimizers, samplers, or hierarchical modeling systems.

The examples in this paper are drawn from cognitive tasks because such tasks motivate struc-

tured response architectures. The software contribution, however, is statistical: **AccumulatR** gives users a compact R interface for declaring those architectures and an R/C++ evaluator for the probabilities they imply (R Core Team 2025; Eddelbuettel and Francois 2011). Section 2 introduces the package objects and public API. Section 3 summarizes the mathematical representation. Section 4 gives the basic R workflow. Section 5 presents structured model examples. Sections 6 and 7 describe likelihood evaluation and implementation details, followed by a comparison to related software and a discussion.

2. Software design

2.1. Objects and workflow

The package distinguishes model structure from numerical parameter values. A `race_spec` object describes the architecture: accumulators, pools, observed outcome rules, components, triggers, mixture settings, and parameter naming rules. A finalized `model_structure` object is produced for simulation, data preparation, response probabilities, and likelihood contexts. The compiled likelihood context is built once and then reused across many parameter evaluations.

Stage	Main functions	Role
Spec.	<code>race_spec()</code> , <code>add_accumulator()</code> , <code>add_pool()</code> , <code>add_outcome()</code>	Declare the latent accumulators, pooled sources, and observed response labels.
Rules	<code>first_of()</code> , <code>all_of()</code> , <code>none_of()</code> , <code>inhibit()</code> , <code>after()</code> , <code>add_trigger()</code>	Express logical outcome rules, blocking rules, staged onsets, and shared absence events.
Params	<code>set_parameters()</code> , <code>par_names()</code> , <code>build_param_matrix()</code>	Control public parameter names and expand named values to trial-wise parameter rows.
Mixing	<code>add_component()</code> , <code>set_mixture()</code>	Define component membership and specify whether mixture weights are fixed or estimated as sampled-mixture parameters.
Runtime	<code>finalize_model()</code> , <code>simulate()</code> , <code>prepare_data()</code> , <code>make_context()</code> , <code>log_likelihood()</code>	Finalize a specification, simulate data, prepare observations, and evaluate the likelihood.
Inspect	<code>processing_tree()</code> , <code>response_probabilities()</code> , <code>complexity_metrics()</code>	Inspect response logic, marginal response probabilities, and compiled complexity diagnostics.

Table 1: Main public functions in **AccumulatR**.

The workflow in Table 1 is deliberately explicit. Model-building functions return modified

specifications, so they can be chained with the base R pipe. Parameter values are not stored in the model object. Instead `build_param_matrix()` expands a named vector into the rectangular or long parameter layout expected by the simulator and likelihood evaluator. This separation lets the same model architecture be evaluated under many trial-wise parameter sets.

2.2. Parameter naming

Parameter names are public interface, not an implementation detail. By default, compatible accumulator parameters are grouped. For example, a two-accumulator lognormal race has public parameters `m`, `s`, and `t0`; both accumulators share those values unless the specification requests otherwise. The function `set_parameters()` changes this mapping. Its `separate` argument splits grouped parameters, `share` intentionally maps several internal parameters to one new public name, and `rename` changes a public name without merging distinct parameters. The result can be inspected with `par_names()`.

This design avoids the common ambiguity between an accumulator-level parameter and a public parameter. The name `left.m` is valid only if the specification has separated `m` for the accumulator `left`. Without that declaration, the valid name is `m`. The same mechanism is used for models that combine distribution families. If the same suffix has incompatible meanings across distributions, the default public name is qualified by distribution, for example `lognormal.s` and `RDM.s`.

2.3. Components and mixtures

Components represent qualitatively different process variants within one model. A component declaration defines membership only: it says which accumulators are active in a component, and optionally how many ordered responses that component records. Mixture probabilities are configured separately with `set_mixture()`. In `mode = "fixed"`, component weights are supplied as known probabilities. In `mode = "sample"`, non-reference components expose probability parameters such as `p.fast`; the reference receives the residual probability.

The data determine whether a component is observed on a trial. If a `component` column is present, likelihood evaluation conditions on that component. If the column is absent or contains missing component labels, the likelihood marginalizes over components using the model's mixture specification. The same convention is used by `response_probabilities()`.

3. Model representation

3.1. Leaves, sources, and response rules

The basic latent object is a finishing time. For a leaf accumulator a , let T_a denote its finishing time, with distribution function F_a , density f_a , and survival function S_a :

$$F_a(t) = \Pr(T_a \leq t), \quad f_a(t) = F'_a(t), \quad S_a(t) = 1 - F_a(t). \quad (1)$$

A source is either a leaf or a pool. An outcome rule defines when an observed response becomes available. The simplest outcome rule is a direct readout from one source. More complex rules use first-of, all-of, absence, and inhibition operators.

Distribution	Public parameters	Notes
lognormal	<code>m</code> , <code>s</code>	Log-scale location and spread.
gamma	<code>shape</code> , <code>rate</code>	Gamma finishing-time distribution.
exgauss	<code>mu</code> , <code>sigma</code> , <code>tau</code>	Gaussian plus exponential finishing time.
LBA	<code>v</code> , <code>B</code> , <code>A</code> , <code>sv</code>	Linear ballistic accumulator.
RDM	<code>v</code> , <code>B</code> , <code>A</code> , <code>s</code>	Race diffusion model parameterization.

Table 2: Accumulator distributions currently available in **AccumulatR**. All accumulators also have the non-decision-time parameter `t0`, which defaults to zero if omitted.

For a simple independent race in which response r is observed at time t , the familiar density is

$$\ell_r(t) = f_r(t) \prod_{j \neq r} S_j(t). \quad (2)$$

AccumulatR treats Equation 2 as a special case of a more general order-region representation. A target outcome may become true through several transition scenarios, and each scenario is combined with the condition that no competing observed outcome has occurred earlier. For direct, independent races this lowers immediately to products of densities and survival functions. For shared, nested, or guarded rules the evaluator keeps the joint order constraints until they can be projected correctly.

3.2. Pools and logical rules

A pool combines several sources into one source. If pool P has members $T_1, \dots, T_n$ and finishes when the k th member finishes, then

$$F_P(t) = \Pr \left(\sum_{j=1}^n 1\{T_j \leq t\} \geq k \right). \quad (3)$$

For independent exchangeable members with common F and $S = 1 - F$, this reduces to

$$F_P(t) = \sum_{m=k}^n \binom{n}{m} F(t)^m S(t)^{n-m}. \quad (4)$$

The software does not require exchangeability; the pool is compiled as order constraints over its members.

The helpers `first_of()` and `all_of()` define disjunction and conjunction over response rules. With independent children, `first_of()` becomes true when the earliest child becomes true, and `all_of()` becomes true when the last required child becomes true:

$$f_{\text{first}}(t) = \sum_{r=1}^m f_r(t) \prod_{q \neq r} S_q(t), \quad f_{\text{all}}(t) = \sum_{r=1}^m f_r(t) \prod_{q \neq r} F_q(t). \quad (5)$$

The helper `none_of()` is an absence condition. The helper `inhibit()` defines a response-generating rule and a blocker. For independent direct sources,

$$f_{\text{inhibit}(A,B)}(t) = f_A(t) S_B(t), \quad (6)$$

but the implemented representation also handles composite blockers and shared sources.

3.3. Onsets, triggers, mixtures, and ranks

An accumulator can have an absolute onset or a chained onset. With an absolute delay d_a , the observed finishing time is $T_a = d_a + T_a^*$. With `after()`, the accumulator begins only after another source has finished. If accumulator a starts after source U with lag δ , then

$$T_a = T_U + \delta + T_a^*. \quad (7)$$

The delayed accumulator then participates in the same outcome rules as any other accumulator.

Triggers control whether accumulators are present on a trial. A trigger is a named absence-probability parameter. All members in one `add_trigger()` call share the same absence draw; separate calls create independent absence draws. If a shared trigger has absence probability q_g , the trial likelihood is a mixture over the trigger-on and trigger-off states:

$$L = (1 - q_g)L^{\text{on}} + q_gL^{\text{off}}. \quad (8)$$

Mixture components use the same likelihood principle. If a component is observed, the trial likelihood conditions on that component. If it is latent, the likelihood averages over components:

$$L_i = \sum_c \pi_{ic} L_{ic}. \quad (9)$$

For ranked-response data, the model can retain more than one ordered response per trial by setting `n_outcomes` in `race_spec()`. The likelihood is then evaluated sequentially over the observed response-time pairs $(r_1, t_1), \dots, (r_K, t_K)$.

4. Using AccumulatR

4.1. A two-choice race

The following example defines a two-choice lognormal race. Before parameter separation, both accumulators share the default public parameters:

```
R> library("AccumulatR")
R> spec <- race_spec() |>
+   add_accumulator("left", "lognormal") |>
+   add_accumulator("right", "lognormal") |>
+   add_outcome("left", "left") |>
+   add_outcome("right", "right")
R> par_names(spec)

[1] "m" "s" "t0"
```

To give the two accumulators separate lognormal locations and spreads, the specification uses `set_parameters()`. The non-decision time `t0` remains shared and is not supplied below, so it defaults to zero.

```
R> model <- spec />
+   set_parameters(separate = list(m = TRUE, s = TRUE)) />
+   finalize_model()
R> pars <- c(
+   left.m = log(0.28), left.s = 0.16,
+   right.m = log(0.35), right.s = 0.18
+ )
R> param_df <- build_param_matrix(model, pars, n_trials = 120)
R> sim <- simulate(model, param_df, seed = 20260612)
R> head(sim, 4)
```

```
  trials      R      rt
1      1 right 0.3167664
2      2 left 0.2847782
3      3 left 0.3259237
4      4 left 0.2515444
```

The simulated data use the same columns expected for real behavioral data: `trials`, `R`, and `rt`. The likelihood workflow prepares the data once, builds a compiled context once, and evaluates parameter matrices as needed.

```
R> prepared <- prepare_data(model, sim[c("trials", "R", "rt")])
R> ctx <- make_context(model)
R> log_likelihood(ctx, prepared,
+   build_param_matrix(model, pars, trial_df = prepared))
```

```
[1] 157.0651
```

Response probabilities can be computed without simulation:

```
R> one_trial <- build_param_matrix(model, pars, n_trials = 1)
R> round(response_probabilities(model, one_trial), 3)
```

```
  left right
0.823 0.177
```

The value returned by `log_likelihood()` is a density-based log-likelihood, so it need not be negative. With continuous response-time models, densities can be greater than one.

4.2. Trial-wise parameters

The parameter matrix is trial-wise by construction. A typical fitting function therefore takes a low-dimensional vector of free parameters, maps it to the public parameter names of the model, and then calls `build_param_matrix()` with the prepared data. For example, a minimal negative log-likelihood function for the two-choice race is:

```

neg_loglik <- function(theta) {
  est <- c(
    left.m = theta[["left.m"]],
    left.s = exp(theta[["log_left.s"]]),
    right.m = theta[["right.m"]],
    right.s = exp(theta[["log_right.s"]])
  )
  params <- build_param_matrix(model, est, trial_df = prepared)
  -as.numeric(log_likelihood(ctx, prepared, params))
}

```

The package does not impose an optimizer or sampler. The likelihood can be used with `optim()`, custom maximum-likelihood routines, or external Bayesian engines that call back into R.

5. Structured model examples

5.1. Pooled evidence

Pools represent responses generated by a threshold number of latent channels. In the following specification, response A is observed when any two of three A channels have finished, while response B is generated by a single competing accumulator.

```

pool_model <- race_spec() |>
  add_accumulator("A1", "lognormal") |>
  add_accumulator("A2", "lognormal") |>
  add_accumulator("A3", "lognormal") |>
  add_accumulator("B", "lognormal") |>
  add_pool("A_pool", c("A1", "A2", "A3"), k = 2L) |>
  add_outcome("A", "A_pool") |>
  add_outcome("B", "B") |>
  set_parameters(separate = list(m = TRUE, s = TRUE)) |>
  finalize_model()

```

When A is observed, its response time is the finishing time of the second finisher in `A_pool`. The likelihood uses that pooled completion rule directly; the user does not need to derive a separate density for the order statistic.

5.2. Logical response rules

Observed responses can be defined by logical rules rather than by single accumulators. The next model says that a response occurs only if an encoding stage has finished, either choice route has finished, and the stop process has not occurred.

```

logical_model <- race_spec() |>
  add_accumulator("encoding", "lognormal") |>

```

```

add_accumulator("left_route", "lognormal") |>
add_accumulator("right_route", "lognormal") |>
add_accumulator("stop", "lognormal") |>
add_outcome(
  "choice",
  all_of(
    "encoding",
    first_of("left_route", "right_route"),
    none_of("stop")
  )
) |>
finalize_model()

```

For stop-like architectures where one process generates a response and another can block it, `inhibit()` gives a more direct expression:

```

stop_model <- race_spec() |>
  add_accumulator("go", "lognormal") |>
  add_accumulator("stop", "lognormal") |>
  add_outcome("respond", inhibit("go", "stop")) |>
  finalize_model()

```

Both examples use string labels directly. The helper `build_outcome_expr()` is available for programmatic builders that need quoted R expressions.

5.3. Chained onsets and shared triggers

Chained onsets express staged processing. Here accumulator C starts only after latent accumulator B has finished:

```

onset_model <- race_spec() |>
  add_accumulator("A", "lognormal") |>
  add_accumulator("B", "lognormal") |>
  add_accumulator("C", "lognormal", onset = after("B")) |>
  add_outcome("A", "A") |>
  add_outcome("C", "C") |>
  set_parameters(separate = list(m = TRUE, s = TRUE)) |>
  finalize_model()

```

Triggers express shared absence events. In the next model, the two `go` accumulators share one absence draw and one absence-probability parameter:

```

trigger_model <- race_spec() |>
  add_accumulator("go1", "lognormal") |>
  add_accumulator("go2", "lognormal") |>
  add_outcome("R1", "go1") |>
  add_outcome("R2", "go2") |>

```

```

add_trigger("shared_trigger", members = c("go1", "go2")) |>
set_parameters(separate = list(m = TRUE, s = TRUE)) |>
finalize_model()

```

If separate `add_trigger()` calls are used instead, the absence draws are independent. Their probabilities can still be given a shared public parameter through `set_parameters()`; that changes parameter naming but not the independence of the trigger draws.

5.4. Latent mixtures

Components are useful when trials may come from qualitatively different processing modes. The following model has a fast and a slow target route, with one competitor. The component declarations define which accumulators are active in each component, and `set_mixture()` declares the component weights to be sampled parameters.

```

mix_model <- race_spec() |>
  add_accumulator("target_fast", "lognormal") |>
  add_accumulator("target_slow", "lognormal") |>
  add_accumulator("competitor", "lognormal") |>
  add_pool("target", c("target_fast", "target_slow")) |>
  add_outcome("R1", "target") |>
  add_outcome("R2", "competitor") |>
  add_component("fast", members = c("target_fast", "competitor")) |>
  add_component("slow", members = c("target_slow", "competitor")) |>
  set_parameters(separate = list(m = TRUE, s = TRUE)) |>
  set_mixture(mode = "sample", reference = "slow") |>
  finalize_model()

```

For this model, the public parameter `p.fast` is the probability of the fast component and the slow component receives the residual mass. If the data include a `component` column, the likelihood conditions on the observed component. Without that column, `fast` and `slow` are treated as latent processing modes and marginalized.

5.5. Ranked responses

Some experiments record more than one ordered response on a trial. Setting `n_outcomes = 2L` asks the simulator and likelihood evaluator to retain the first and second finishing responses:

```

rank_model <- race_spec(n_outcomes = 2L) |>
  add_accumulator("A", "lognormal") |>
  add_accumulator("B", "lognormal") |>
  add_outcome("A", "A") |>
  add_outcome("B", "B") |>
  set_parameters(separate = list(m = TRUE, s = TRUE)) |>
  finalize_model()

```

The corresponding data columns are `R`, `rt`, `R2`, and `rt2`. Higher ranks follow the same naming pattern.

6. Likelihood evaluation and fitting

The likelihood interface is intentionally narrow. `prepare_data()` checks and reshapes behavioral observations. `make_context()` compiles the model-side runtime state. The likelihood call combines a context, prepared data, and a parameter matrix. This separation matters for estimation because model compilation is usually more expensive than evaluating one new parameter matrix.

```
prepared <- prepare_data(model, data_df)
ctx <- make_context(model)

objective <- function(theta) {
  values <- transform_theta_to_named_parameters(theta)
  params <- build_param_matrix(model, values, trial_df = prepared)
  -as.numeric(log_likelihood(ctx, prepared, params))
}

fit <- optim(start, objective, method = "Nelder-Mead")
```

The transformation placeholder in this example is user code: it maps unconstrained optimization variables to the public parameter names of the model. For example, positive spread, boundary, and non-decision-time parameters are commonly optimized on the log scale and exponentiated before calling `build_param_matrix()`. This keeps the package focused on model evaluation while leaving estimation strategy to the analysis.

The parameter matrix may be rectangular, with one row per accumulator per trial, or long, with rows aligned to prepared data for component-specific architectures. The `layout = "auto"` default selects the appropriate layout for most uses. Prepared data can also be compressed when repeated observations share the same prepared representation; in that case the likelihood sums weighted compact rows and expands trial-wise values when requested.

7. Implementation

The implementation has three layers. The first layer is the R specification interface. It validates names, stores accumulator definitions, records outcome expressions, normalizes parameter mappings, and finalizes a model into a runtime-oriented structure. The second layer prepares data and parameters for repeated evaluation. The third layer is the compiled evaluator, which receives the finalized model plan and performs simulation or likelihood calculations.

The exact likelihood evaluator lowers outcome rules into transition scenarios. A scenario records which latent source releases a target rule at the observed time, which other sources must already be available, and which blockers or competitors must not have occurred earlier. These statements are represented as order constraints over latent finishing times. The compiler simplifies contradictory or redundant cells, projects simple constraints to density,

distribution, or survival factors, and retains numerical integration only where the order structure cannot be reduced further.

This representation is important for models with shared sources. If two observed outcomes both depend on a common latent stage, they can tie with positive density because the same latent finishing time releases both rules. Treating every tie as a zero-probability boundary would be incorrect. By keeping shared latent identity in the region representation, **AccumulatR** can distinguish ordinary zero-measure equality from model-induced equality.

Most computational cost that depends only on model structure is paid when the context is built. Repeated likelihood calls reuse the compiled plan and mostly pay the cost of evaluating densities, probabilities, and remaining integrals under new parameter values. For complicated architectures the number of symbolic cells can grow quickly. When a context is built with `diagnostics = TRUE`, `complexity_metrics()` reports the compiled complexity so the user can inspect whether a specification is becoming computationally expensive.

8. Relation to existing software

AccumulatR is not a replacement for specialized diffusion-model packages, model-specific fitters, or full hierarchical estimation systems. The relevant software landscape is broad, and several packages span more than one role. For the present comparison the useful distinction is between the statistical object handled by a package and the level of workflow it exposes.

The distinguishing feature of **AccumulatR** is not that it is more general than all of these systems. It is more specific. Standard race models already provide a flexible alternative to two-threshold diffusion models because they represent several latent finishing times rather than a single relative-evidence process. However, most software for race models still treats the mapping from latent finishing times to observed responses as part of the named model: for example, the first accumulator to finish wins, or a fixed LBA/RDM/lognormal-race readout is assumed. **AccumulatR** makes that readout architecture explicit. The same object can represent ordinary races, pooled accumulators, staged routes, latent mixtures, guarded responses, shared absence events, and ranked responses, and the same object is used for simulation, inspection, response-probability calculation, and likelihood evaluation.

There are corresponding limitations. The package currently assumes that leaf accumulators are conditionally independent given parameters, components, and trigger states. It supplies likelihood evaluation but not a complete hierarchical modeling language. Users who need group-level priors, posterior sampling diagnostics, or formula-based regression over parameters must supply that estimation layer separately. In exchange, they obtain a model object whose architecture can be manipulated directly in R.

Table 3 summarizes this classification. The rows classify packages by their most relevant role for the present comparison, not by every feature they expose. Some packages span categories; for example, **HSSM** supports DDMs and broader SSMs, but is listed with broad EAM estimation because its current user-facing role is hierarchical estimation with simulator and amortized-likelihood infrastructure.

9. Replication material

The manuscript directory contains executable R scripts under `dev/LatexFiles/examples`.

Category	Examples	Relation to AccumulatR
DDM/two-threshold evaluation	RWiener , fddm , ddModel , ream , PyDDM	Evaluate or simulate named DDM, generalized DDM, or two-threshold accumulation processes. The response mapping is usually fixed by the two-boundary process.
Race/broader evaluation	rtdists , lbaModel , CircularDDM , dynConfR , eam , ssms , SequentialSamplingModels.jl	Provide densities, simulators, or prediction functions for LBA, race, continuous-response, confidence, or broader SSM families. The response rule is usually the one built into the named model.
DDM/two-threshold estimation	fast-dm , dRiftDM , DMCfun , diffIRT , RegDDM , brms , bmm , HDDM , PyBEAM , PyDDM	Estimate DDMs, two-threshold models, task-specific DDM extensions, or hierarchical regressions.
Race/broad EAM estimation	glba , DMC/ggdmc , EMC2 , pmwg , eam , HSSM	Estimate LBA, RDM, lognormal-race, and other SSM families, often using hierarchical, simulator-based, or amortized-likelihood workflows. AccumulatR supplies the architecture-specific likelihood object that such workflows otherwise need to hand-code.

Table 3: Related software for evidence accumulation and sequential-sampling models, grouped by model family and workflow role.

The script `01-basic-workflow.R` reproduces the two-choice race in Section 4, including the reported response probabilities. The script `02-structured-models.R` constructs pooled, logical, trigger, onset, and ranked-response examples. The script `03-mixtures-and-fitting.R` constructs the sampled-mixture example and runs a small likelihood-based estimate of the mixture weight. The top-level `code.R` script sources the examples and prints `sessionInfo()`, which is the form expected for JSS replication material.

10. Discussion

AccumulatR provides a declarative interface for a class of structured evidence-accumulation models. Its core design choice is to make architecture explicit: accumulators, pools, outcome rules, triggers, components, and ranked readouts are part of the model object rather than ad hoc code around a fixed density. The same object then supports simulation, response-probability calculation, data preparation, and likelihood evaluation.

This architecture-first design is most useful for analyses where the main scientific hypothesis concerns process structure. It also keeps the package honest about what it does not do. **AccumulatR** evaluates models; it does not attempt to be a universal optimizer, sampler, or hierarchical modeling system. That boundary keeps the interface small and makes it easier to combine the package with external estimation tools.

The most important future work is broader empirical demonstration and a richer estimation interface around the existing likelihood engine. Additional finishing-time families, more diag-

nostic summaries, and tighter integration with hierarchical workflows would also be natural extensions. The present software already supplies the central machinery needed for those extensions: a compact model language for structured accumulator architectures and a reusable compiled evaluator for the likelihoods implied by those architectures.

Acknowledgments

The package and manuscript benefited from the published literature on sequential-sampling models and from the *Journal of Statistical Software* style and replication guidelines.

References

- Bürkner PC (2017). “Brms: An R Package for Bayesian Multilevel Models Using Stan.” *Journal of Statistical Software*, **80**, 1–28. ISSN 1548-7660. doi:10.18637/jss.v080.i01.
- Cooper G, Innes R, Kuhne C, Cavallaro JP, Gunawan D, Hawkins G, Brown S, Stevenson N (2024). **pmwg**: *Particle Metropolis Within Gibbs*. doi:10.32614/CRAN.package.pmwg. R package version 0.2.7, URL <https://CRAN.R-project.org/package=pmwg>.
- Eddelbuettel D, Francois R (2011). “**Rcpp**: Seamless R and C++ Integration.” *Journal of Statistical Software*, **40**(8), 1–18. doi:10.18637/jss.v040.i08.
- Fengler A, Govindarajan LN, Chen T, Frank MJ (2021). “Likelihood Approximation Networks (LANs) for Fast Inference of Simulation Models in Cognitive Neuroscience.” *eLife*, **10**, e65074. ISSN 2050-084X. doi:10.7554/eLife.65074.
- Fengler A, Xu Y, Bera K, Omar A, Frank MJ (2026). **HSSM**: *Hierarchical Sequential Sampling Modeling*. Python package documentation, URL <https://lnccbrown.github.io/HSSM/>.
- Fernandez K, Makowski D, Fisher C (2024). “**SequentialSamplingModels.jl**: Simulating and Evaluating Cognitive Models of Response Times in Julia.” ArXiv:2411.06631, URL <https://arxiv.org/abs/2411.06631>.
- Foster KB, Singmann H, Zeileis A (2024). **fddm**: *Fast Implementation of the Diffusion Decision Model*. doi:10.32614/CRAN.package.fddm. R package version 1.0-2, URL <https://CRAN.R-project.org/package=fddm>.
- Hartmann R, Murrow M (2025). **ream**: *Density, Distribution, and Sampling Functions for Evidence Accumulation Models*. doi:10.32614/CRAN.package.ream. R package version 1.0-10, URL <https://CRAN.R-project.org/package=ream>.
- Heathcote A, Lin YS, Reynolds A, Strickland L, Gretton M, Matzke D (2019). “Dynamic Models of Choice.” *Behavior Research Methods*, **51**(2), 961–985. ISSN 1554-3528. doi:10.3758/s13428-018-1067-y.
- Heathcote A, Matzke D (2022). “Winner Takes All! What Are Race Models, and Why and How Should Psychologists Use Them?” *Current Directions in Psychological Science*, **31**(5), 383–394. ISSN 0963-7214. doi:10.1177/09637214221095852.

- Hellmann S, Rausch M (2025). **dynConfiR**: *Dynamic Models for Confidence and Response Time Distributions*. doi:10.32614/CRAN.package.dynConfiR. R package version 1.1.1, URL <https://CRAN.R-project.org/package=dynConfiR>.
- Jin Z (2025). **RegDDM**: *Generalized Linear Regression with DDM*. doi:10.32614/CRAN.package.RegDDM. R package version 1.1, URL <https://CRAN.R-project.org/package=RegDDM>.
- Koob V, Richter T, Janczyk M (2026). **dRiftDM**: *Estimating Time-Dependent Drift Diffusion Models*. doi:10.32614/CRAN.package.dRiftDM. R package version 0.3.2, URL <https://CRAN.R-project.org/package=dRiftDM>.
- Lin YS (2025a). **ddModel**: *The Decision Diffusion Model*. doi:10.32614/CRAN.package.ddModel. R package version 0.2.9.0, URL <https://CRAN.R-project.org/package=ddModel>.
- Lin YS (2025b). **lbaModel**: *The Linear Ballistic Accumulation Model*. doi:10.32614/CRAN.package.lbaModel. R package version 0.2.9.2, URL <https://CRAN.R-project.org/package=lbaModel>.
- Lin YS, Heathcote A (2025). **ggdmc**: *Cognitive Models*. doi:10.32614/CRAN.package.ggdmc. R package version 0.2.6.2, URL <https://CRAN.R-project.org/package=ggdmc>.
- Lin YS, Heathcote A, Kvam P (2018). **CircularDDM**: *Circular Drift-Diffusion Model*. doi:10.32614/CRAN.package.CircularDDM. R package version 0.1.0, URL <https://CRAN.R-project.org/package=CircularDDM>.
- Liu Y (2007). “Queueing Network Modeling of Mental Architecture, Response Time, and Response Accuracy: Reflected Multidimensional Diffusions.” In *Proceedings of ICCM 2007: Eighth International Conference on Cognitive Modeling*, pp. 285–290. Taylor & Francis/Psychology Press, Oxford, UK.
- Mackenzie IG, Dudschig C (2024). **DMCfun**: *Diffusion Model of Conflict in Reaction Time Tasks*. doi:10.32614/CRAN.package.DMCfun. R package version 4.0.1, URL <https://CRAN.R-project.org/package=DMCfun>.
- Molenaar D, Tuerlinckx F, van der Maas HLJ (2015). “Fitting Diffusion Item Response Theory Models for Responses and Response Times Using the R Package **diffIRT**.” *Journal of Statistical Software*, **66**(4), 1–34. doi:10.18637/jss.v066.i04.
- Murrow M, Holmes WR (2024). “**PyBEAM**: A Bayesian Approach to Parameter Inference for a Wide Class of Binary Evidence Accumulation Models.” *Behavior Research Methods*, **56**(3), 2636–2656. doi:10.3758/s13428-023-02162-w.
- Popov V, Frischkorn GT, Li C, Buerkner PC (2026). **bmm**: *Easy and Accessible Bayesian Measurement Models Using brms*. doi:10.32614/CRAN.package.bmm. R package version 1.3.1, URL <https://CRAN.R-project.org/package=bmm>.
- R Core Team (2025). *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria. URL <https://www.R-project.org/>.

- Schweickert R (1978). “A Critical Path Generalization of the Additive Factor Method: Analysis of a Stroop Task.” *Journal of Mathematical Psychology*, **18**, 105–139.
- Shinn M, Lam NH, Murray JD (2020). “A Flexible Framework for Simulating and Fitting Generalized Drift-Diffusion Models.” *eLife*, **9**, e56938. doi:10.7554/eLife.56938.
- Singmann H, Brown S, Gretton M, Heathcote A, Voss A, Voss J, Terry A (2026). *rtdist: Response Time Distributions*. doi:10.32614/CRAN.package.rtdists. R package version 0.11-6, URL <https://CRAN.R-project.org/package=rtdists>.
- Stevenson N, Donzallaz MC, Innes R, Forstmann B, Matzke D, Heathcote A (2024). “EMC2: An R Package for Cognitive Models of Choice.” doi:10.31234/osf.io/2e4dq.
- Townsend JT, Wenger MJ (2004). “The Serial–Parallel Dilemma: A Case Study in a Linkage of Theory and Method.” *Psychonomic Bulletin & Review*, **11**(3), 391–418. doi:10.3758/BF03196588.
- Voss A, Voss J (2007). “fast-dm: A Free Program for Efficient Diffusion Model Analysis.” *Behavior Research Methods*, **39**(4), 767–775. doi:10.3758/BF03192967.
- Wabersich D, Vandekerckhove J (2014). “The RWiener Package: An R Package Providing Distribution Functions for the Wiener Diffusion Model.” *The R Journal*, **6**(1), 49–56. ISSN 2073-4859.
- Wiecki TV, Sofer I, Frank MJ (2013). “HDDM: Hierarchical Bayesian Estimation of the Drift-Diffusion Model in Python.” *Frontiers in Neuroinformatics*, **7**, 14. doi:10.3389/fninf.2013.00014.
- Zhu G, Yang G (2026). *eam: Evidence Accumulation Models*. doi:10.32614/CRAN.package.eam. R package version 1.2.2, URL <https://CRAN.R-project.org/package=eam>.

Affiliation:

Niek Stevenson

Independent researcher

E-mail: niek.stevenson@gmail.com

URL: <https://github.com/niekstevenson/AccumulatR/>

Journal of Statistical Software

published by the Foundation for Open Access Statistics

MMMMMM YYYY, Volume VV, Issue II

doi:10.18637/jss.v000.i00

<http://www.jstatsoft.org/>

<http://www.foastat.org/>

Submitted: yyyy-mm-dd

Accepted: yyyy-mm-dd